

Cellular Automata Project

Documentation: Conway's Game of Life & Langton's Ant

1. Introduction and Theoretical Background

Cellular Automata (CA) are discrete dynamical systems that serve as powerful mathematical models in computability theory, physics, and complex systems. Formally, a cellular automaton consists of a regular grid of cells, each residing in one of a finite number of states. The system evolves through discrete time steps ($t = 0, 1, 2, \dots$), where the state of each cell at time $t + 1$ is determined uniquely by its own state and the states of its neighboring cells at time t , according to a local transition function.

1.1 Formal Mathematical Formulation

Let $\mathcal{L}$ represent an $N \times N$ two-dimensional grid of cells. For any cell $c = (i, j) \in \mathcal{L}$, its state at time t is denoted by:

$$s_t(c) \in S$$

For binary cellular automata, such as Conway's Game of Life, the state space is $S = \{0, 1\}$, where 0 represents a dead cell and 1 represents a living cell.

The state transition is dictated by a local transition function δ :

$$\delta : S \times S^k \rightarrow S$$

where k represents the number of neighbors in the designated neighborhood. For a Moore neighborhood (8-connectivity), $k = 8$, and the neighborhood of a cell $c = (i, j)$ is defined as:

$$\text{Neigh}(c) = \{(i + u, j + v) \mid u, v \in \{-1, 0, 1\} \setminus \{(0, 0)\}\}$$

Thus, the state of cell c at the next time step $t + 1$ is updated based on its current state and the sum of its living neighbors:

$$s_{t+1}(c) = \delta \left(s_t(c), \sum_{n \in \text{Neigh}(c)} s_t(n) \right)$$

In a synchronous cellular automaton, this update rule is applied to all cells in the grid simultaneously.

1.2 Emergence and Turing Completeness

One of the most profound characteristics of cellular automata is emergence: the manifestation of complex, self-organizing global patterns and behaviors from extremely simple, localized rules.

Furthermore, certain cellular automata, including Conway's Game of Life, are Turing Complete. This means they can simulate any Turing machine and, consequently, any computable algorithm. In Conway's Game of Life, Turing completeness is proven by constructing physical-like representations of digital logic gates (AND, OR, NOT, etc.), memory registers, and signal transmission wires using moving gliders and static blocks.

2. Part 1: Conway's Game of Life

Conway's Game of Life (GoL) is the most famous 2D cellular automaton. It operates on a grid of cells using a Moore neighborhood and relies on four transition rules.

2.1 The Four Transition Rules

At each time step, the following rules determine the next state of a cell:

1. Underpopulation: Any live cell ($s_t(c) = 1$) with fewer than 2 live neighbors dies ($s_{t+1}(c) = 0$).
2. Survival: Any live cell ($s_t(c) = 1$) with 2 or 3 live neighbors lives on ($s_{t+1}(c) = 1$).
3. Overpopulation: Any live cell ($s_t(c) = 1$) with more than 3 live neighbors dies ($s_{t+1}(c) = 0$).
4. Reproduction: Any dead cell ($s_t(c) = 0$) with exactly 3 live neighbors becomes a live cell ($s_{t+1}(c) = 1$).

2.2 Core Rules Implementation

When `fastMode` is disabled, the state transition is calculated manually using nested loops.

Boundary Conditions: Finite vs. Toroidal (Wrapping) * Finite Grid: Out-of-bounds neighbors are ignored. The grid boundary acts as a permanent dead zone. *

Toroidal/Wrapping Grid: The grid wraps around. The top row connects to the bottom row, and the leftmost column connects to the rightmost column. This is mathematically handled using modulo arithmetic:

$$\text{Neighbor Index} = (x + \text{offset}) \bmod N$$

Verification: The correctness of the implementation was verified using the Blinker pattern (a period-2 oscillator). Under these rules, a horizontal line of three cells transitions to a vertical line of three cells, and back, indefinitely.

2.3 Gliders: Signal Transmission and Computation

A Glider is the smallest spaceship pattern in Conway's Game of Life. It translates itself diagonally across the grid, shifting by 1 unit along both axes every 4 generations (denoted as a speed of $c/4$).

- Binary Streams: A stream of gliders represents a sequence of bits.
- Information Wires: The paths along which gliders travel act as wires, transporting signals.
- Collisions as Gates: By arranging glider collisions, one can construct digital logic gates (e.g., AND, OR, NOT) to perform computation.

2.4 Gosper Glider Gun Debugging

The Gosper Glider Gun is a periodic oscillator that continuously emits gliders. The initial implementation required precise coordinate alignment. The left block must be perfectly aligned with the left shuttle to prevent destructive interference. With correctly calibrated offsets, the shuttle interacts with the blocks to launch gliders diagonally off the screen indefinitely.

2.5 RLE & Plaintext Parser Implementation

To load complex patterns from external libraries, the system supports: 1.

Plaintext (.cells): Lines starting with `!` are comments; `.` is dead, `0` is alive.

2. Run Length Encoded (.rle): A compressed format using `b` for dead, `o` for alive, `$` for new rows, and runs (e.g., `3o`) for frequency.

2.6 Vectorized Acceleration via 2D Convolution

For large grids ($N > 1024$), calculating states cell-by-cell is computationally expensive. Since neighbor counting is a local, translation-invariant operation, it can be modeled as a spatial 2D Convolution (*).

Let G be the binary grid and K be the Moore neighborhood kernel:

$$K = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

The matrix of neighbor counts C is computed via:

$$C = G * K$$

Using high-performance convolution libraries, the next state is determined by: *

Birth: `(grid == 0) & (neighbor_count == 3)` * Survival: `(grid == 1) & ((neighbor_count == 2) | (neighbor_count == 3))`

3. Part 2: Langton's Ant

Langton's Ant is a two-dimensional universal Turing machine defined on a square grid.

3.1 Rules of Langton's Ant

1. At a white cell (State 0): Turn 90° right, flip color to black (State 1), move forward 1 step.
2. At a black cell (State 1): Turn 90° left, flip color to white (State 0), move forward 1 step.

3.2 Simulation Scenarios and Emergent Behaviors

Running the basic two-color simulation reveals three distinct phases: 1.

Simplicity (Steps 1–500): Small, symmetric patterns are formed. 2. Chaotic

Behavior (Steps 500–10,000): The pattern becomes irregular and pseudo-

random. 3. The Highway (Steps 10,000+): The ant enters a repeating loop of 104 steps, constructing a diagonal, infinite "highway." This transition from chaos to order is a key example of emergence.

3.3 Multi-Color Rulesets

By generalizing to M colors, the ant follows transition rules based on a string of directions (e.g., "RLR"). This allows for complex fractal-like symmetries and diverse geometric structures.

4. Analytical Summary and Conclusion

Cellular Automata provide an elegant framework for studying complexity.

Through Conway's Game of Life, we observe how simple local rules can support signal transmission and computational devices. Through Langton's Ant, we witness how deterministic rules generate chaos, which subsequently self-organizes into highly ordered structures. Implementing these models using vectorized convolution demonstrates how physical modeling tasks can be optimized for high-performance simulation.